

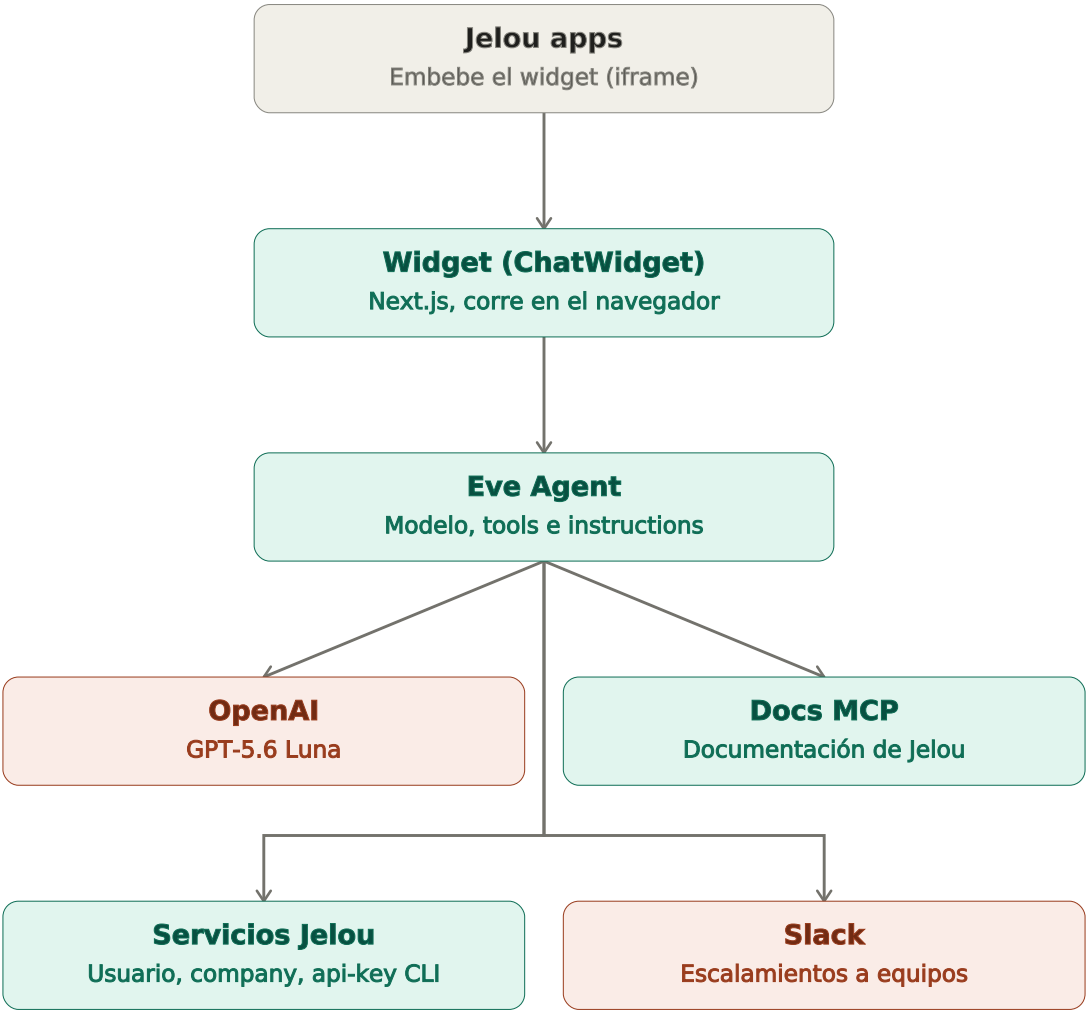
Esquemas del proyecto — jelou-eve-agent

Arquitectura, procesos internos y matriz de tecnologías.

1. Diagrama de arquitectura

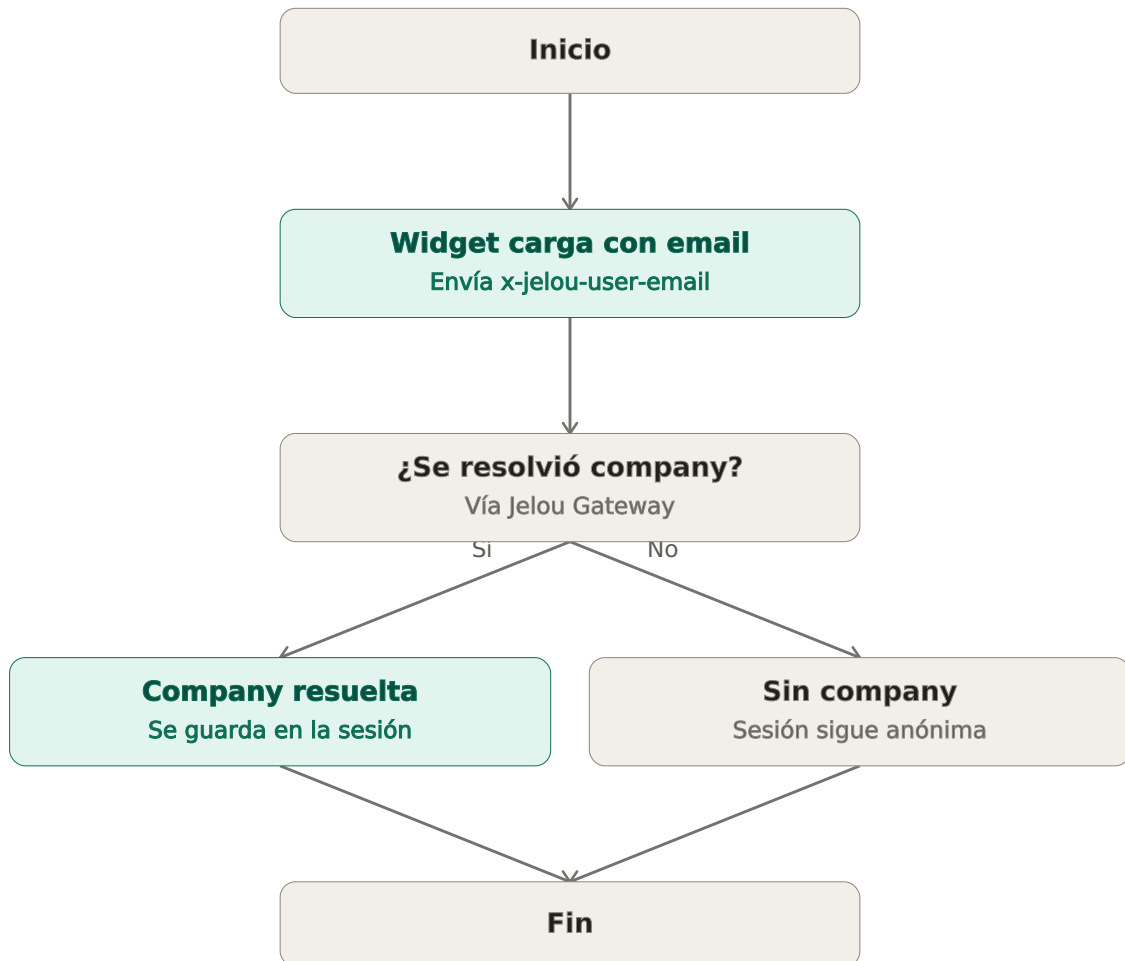
Jelou apps embebe el widget. El widget corre en el navegador y habla con Eve Agent (Next.js + eve framework), que a su vez llama a OpenAI para el modelo, a Docs MCP para documentación, a los servicios internos de Jelou para usuario/company/api-key del CLI, y a Slack para escalamientos.

Colores: gris = componente externo/neutral (fuera del control de Jelou), teal = stack propio de Jelou, coral = proveedor externo que se integra (OpenAI, Slack).



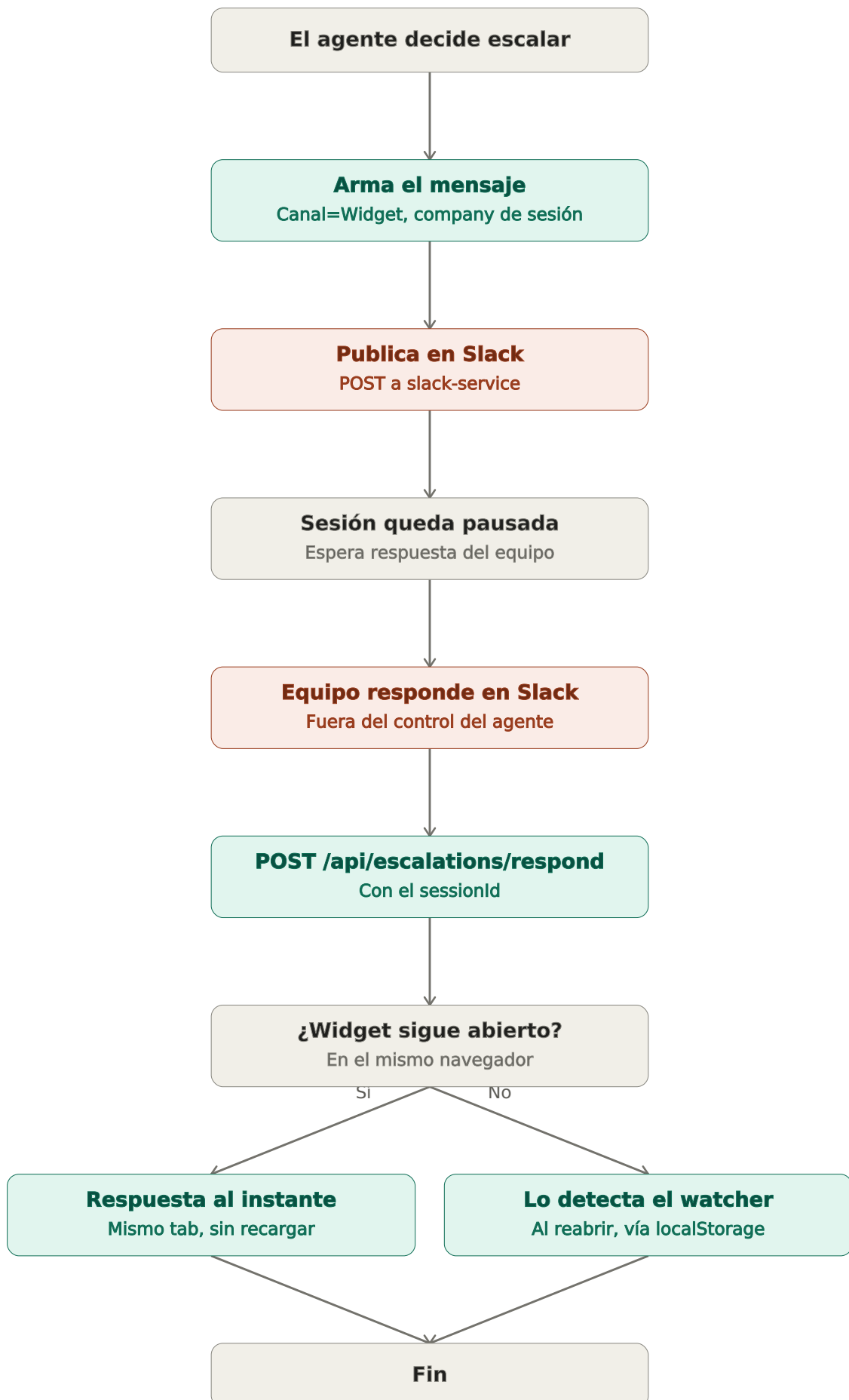
2. Proceso: identificación de usuario y company

El widget carga con el email del visitante y lo manda como header. Eve Agent consulta el gateway de Jelou. Si resuelve una company, la guarda en la sesión y el modelo ya sabe quién es sin preguntar. Si no, la sesión sigue anónima. En ambos casos el chat funciona.



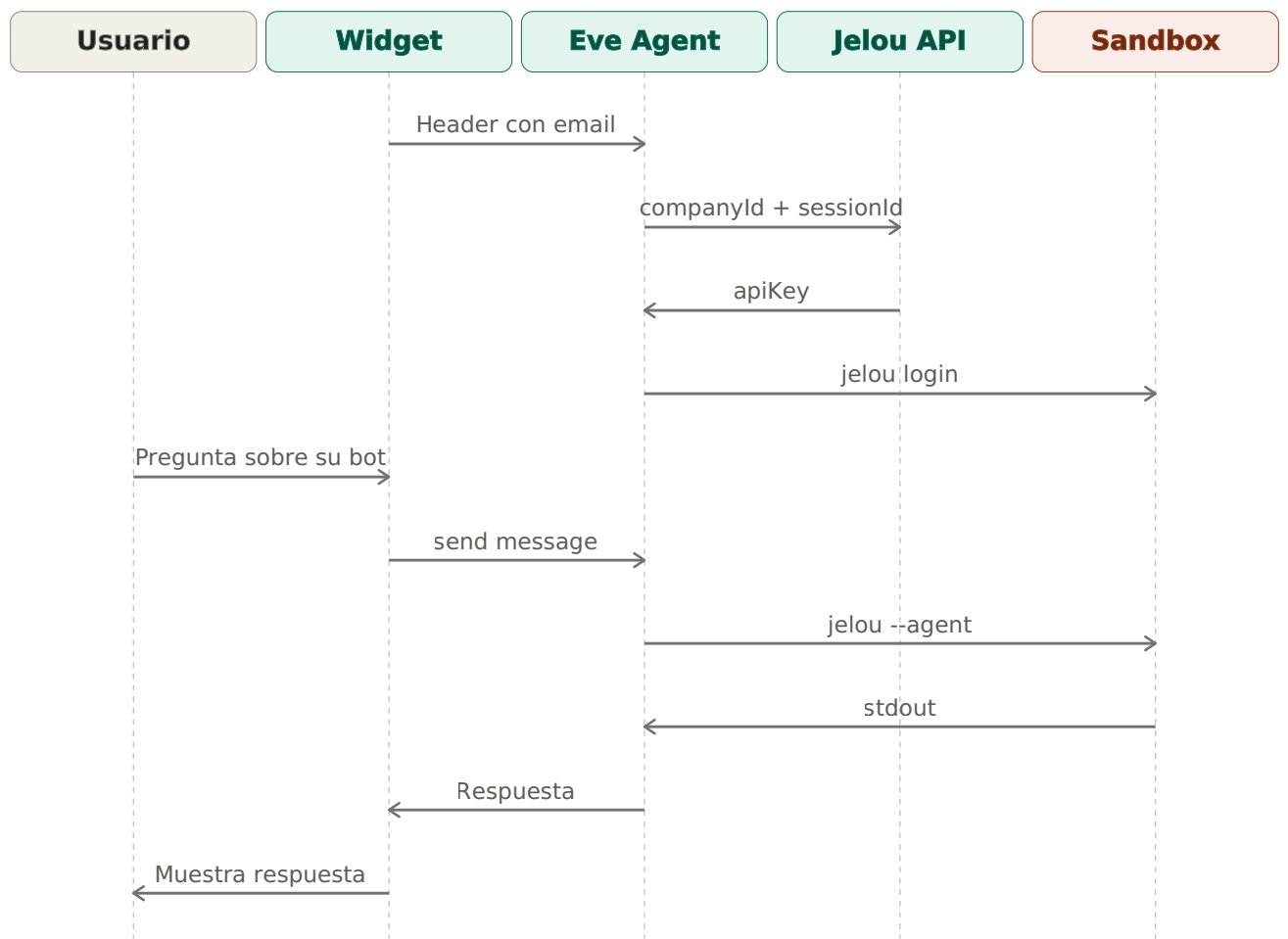
3. Proceso: escalamiento a Slack con pausa y reanudación

El agente arma el mensaje y lo publica en Slack; la sesión queda pausada esperando respuesta del equipo. Cuando el equipo responde, se llama a `/api/escalations/respond` con el `sessionId`, lo que reanuda la sesión. Si el widget sigue abierto se ve al instante; si no, el watcher de reconexión lo detecta al reabrir.



4. Secuencia técnica: analizador con api-key dinámico

Al iniciar sesión, Widget manda el email a Eve Agent, que pide companyId y sessionId a Jelou API y recibe un apiKey, con el que loguea el CLI en el sandbox. Después, cuando el usuario pregunta algo que dispara el analizador, Eve Agent ejecuta el comando en el sandbox y devuelve la respuesta al widget.



5. Matriz de herramientas y tecnologías

Categoría	Herramienta	Uso
Lenguaje	TypeScript	Todo el proyecto (frontend + agente)
Framework web	Next.js 16 (App Router)	Sirve el widget y las páginas de prueba
Runtime del agente	eve framework	Modelo, tools, instructions, sandbox, auth de sesión
Modelo	OpenAI GPT-5.6 Luna (@ai-sdk/openai)	Genera las respuestas del agente

UI	Tailwind CSS + shadcn/ui	Estilos y componentes del widget
Streaming/Markdown	Streamdown	Reveal gradual y render de Markdown en las respuestas
Validación	Zod	Input/output schemas de las tools (<code>escalar</code> , <code>analizador</code>)
Persistencia cliente	localStorage	Historial de chat con TTL de 24h
Identidad de usuario	Jelou Gateway (<code>/platform/v1/utls/users</code>)	Resuelve email → usuario + company
Api-key dinámico del CLI	<code>support-widget-service.fn.jelou.ai</code>	Devuelve (o crea) el api-key por company
CLI	<code>@jelou/cli</code>	Ejecutado por la tool "analizador" (solo lectura)
Sandbox	Vercel Sandbox (<code>eve/sandbox</code>)	Entorno aislado donde corre el CLI
Escalamientos	Servicio de Slack (<code>slack-service-two.vercel.app</code>)	Publica mensajes y permite reanudar sesiones pausadas
Hosting / despliegue	Vercel	Deploy del proyecto y de las envs
Auth deployment-to-deployment	Vercel OIDC	Llamadas TUI/deployment-to-deployment vía eve